

Modifications de Code Backend -- Guide d'Implementation

Projet : Holy Fork -- holly_pi Django backend Date : 2026-07-17 Cible : /Users/antoinemoulin/Documents/Side projects/holly_pi/apps Prerequis : Aucune migration existante ne sera cassee

1. Securiser POST /api/settings/users/ (P0)

Fichier : `apps/settings/views.py`

Ligne 1-11 -- Ajouter les imports :

```
# AVANT (ligne 2)
from rest_framework.permissions import IsAuthenticated

# APRES
from rest_framework.permissions import IsAuthenticated
from apps.staff.permissions import HasPermission
from apps.staff.employee_roles import Permission
from apps.staff.models import RestaurantEmployee
```

Ligne 86-104 -- Modifier UsersSettingsView :

```
# AVANT
class UsersSettingsView(APIView):
    """
    GET /api/settings/users?restaurant_id=X -- liste des utilisateurs
    POST /api/settings/users -- creation utilisateur
    """
    permission_classes = [IsAuthenticated]

    def get(self, request):
        restaurant_id = request.query_params.get('restaurant_id')
        users = HollyUser.objects.all()
        serializer = UserProfileSerializer(users, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = UserRegistrationSerializer(data=request.data)
        if serializer.is_valid():
            user = serializer.save()
            return Response(UserProfileSerializer(user).data, status=201)
        return Response(serializer.errors, status=400)

# APRES
class UsersSettingsView(APIView):
    """
    GET /api/settings/users?restaurant_id=X -- liste des utilisateurs
    POST /api/settings/users -- creation utilisateur (admin only)
    """
    permission_classes = [IsAuthenticated]

    def get_permissions(self):
        if self.request.method == 'POST':
            return [IsAuthenticated(), HasPermission(Permission.MANAGE_STAFF)]
        return [IsAuthenticated()]

    def get(self, request):
        restaurant_id = request.query_params.get('restaurant_id')
```

```

# Filtrer par restaurant si specifié
if restaurant_id:
    employe_user_ids = RestaurantEmploye.objects.filter(
        restaurant_id=restaurant_id
    ).values_list('employe__user_id', flat=True)
    users = HollyUser.objects.filter(id__in=employe_user_ids)
else:
    users = HollyUser.objects.all()

serializer = UserProfileSerializer(users, many=True)
return Response(serializer.data)

def post(self, request):
    serializer = UserRegistrationSerializer(
        data=request.data,
        context={'request': request, 'is_admin_invite': True}
    )
    if serializer.is_valid():
        user = serializer.save()
        return Response(UserProfileSerializer(user).data, status=201)
    return Response(serializer.errors, status=400)

```

Note : Si `HasPermission` n'accepte pas un argument direct, utiliser le pattern `required_permission` sur la classe. Vérifier dans `apps/staff/permissions.py` le constructeur de `HasPermission`.

2. Rendre restaurant_id optionnel dans le serializer (P0)

Fichier : `apps/authentication/serializers.py`

Ligne 51-54 -- Modifier le champ restaurant_id :

```

# AVANT
restaurant_id = serializers.PrimaryKeyRelatedField(
    queryset=Restaurant.objects.all(),
    required=True,
    write_only=True
)

# APRES
restaurant_id = serializers.PrimaryKeyRelatedField(
    queryset=Restaurant.objects.all(),
    required=False,
    allow_null=True,
    write_only=True
)

```

Ligne 65-83 -- Modifier validate() pour gérer restaurant_id absent :

```

# AVANT
def validate(self, attrs):
    if attrs['password'] != attrs['password2']:
        raise serializers.ValidationError({
            "password": "Les mots de passe ne correspondent pas."
        })

    pin_code = attrs['pin_code']
    restaurant_id = attrs['restaurant_id']

    existing_restaurant_employes = RestaurantEmploye.objects.filter(
        restaurant=restaurant_id
    )

```

```

        ).select_related('employee')

    for re in existing_restaurant_employes:
        if re.employee.pin_code and re.employee.pin_code == pin_code:
            raise serializers.ValidationError({
                "pin_code": "Ce PIN code est deja utilise..."
            })
    return attrs

# APRES
def validate(self, attrs):
    if attrs['password'] != attrs['password2']:
        raise serializers.ValidationError({
            "password": "Les mots de passe ne correspondent pas."
        })

    pin_code = attrs['pin_code']
    restaurant = attrs.get('restaurant_id')

    # Valider unicite PIN seulement si un restaurant est specifie
    if restaurant:
        existing_restaurant_employes = RestaurantEmploye.objects.filter(
            restaurant=restaurant
        ).select_related('employee')

        for re in existing_restaurant_employes:
            if re.employee.pin_code and re.employee.pin_code == pin_code:
                raise serializers.ValidationError({
                    "pin_code": "Ce PIN code est deja utilise..."
                })
    return attrs

```

Ligne 86-115 -- Modifier create() pour gerer restaurant_id absent :

```

# AVANT
def create(self, validated_data):
    validated_data.pop('password2', None)
    employee_last_name = validated_data.pop('employee_last_name')
    employee_first_name = validated_data.pop('employee_first_name')
    pin_code = validated_data.pop('pin_code')
    type_employe = validated_data.pop('type_employe_id')
    restaurant = validated_data.pop('restaurant_id')

    user = User.objects.create_user(
        username=validated_data['username'],
        email=validated_data['email'],
        password=validated_data['password'],
        first_name=validated_data.get('first_name', ''),
        last_name=validated_data.get('last_name', '')
    )

    employe = Employee.objects.create(
        user=user,
        nom=employee_last_name,
        prenom=employee_first_name,
        pin_code=pin_code,
        type_employe=type_employe
    )

    RestaurantEmploye.objects.create(
        restaurant=restaurant,
        employe=employe
    )
    return user

# APRES

```

```

def create(self, validated_data):
    validated_data.pop('password2', None)
    employee_last_name = validated_data.pop('employee_last_name')
    employee_first_name = validated_data.pop('employee_first_name')
    pin_code = validated_data.pop('pin_code')
    type_employe = validated_data.pop('type_employe_id')
    restaurant = validated_data.pop('restaurant_id', None)

    user = User.objects.create_user(
        username=validated_data['username'],
        email=validated_data['email'],
        password=validated_data['password'],
        first_name=validated_data.get('first_name', ''),
        last_name=validated_data.get('last_name', '')
    )

    employe = Employee.objects.create(
        user=user,
        nom=employee_last_name,
        prenom=employee_first_name,
        pin_code=pin_code,
        type_employe=type_employe
    )

    # Associer au restaurant seulement si specifie
    if restaurant:
        RestaurantEmploye.objects.create(
            restaurant=restaurant,
            employe=employe
        )
    return user

```

3. Ajouter is_active au serializer de mise a jour user (P1)

Fichier : `apps/authentication/serializers.py`

Apres la classe UserProfileSerializer (ligne 136), ajouter :

```

class AdminUserUpdateSerializer(serializers.ModelSerializer):
    """
    Serializer pour la mise a jour admin d'un utilisateur.
    Permet de modifier is_active (activer/desactiver un compte).
    """
    class Meta:
        model = User
        fields = ('id', 'username', 'email', 'first_name', 'last_name',
                  'date_joined', 'is_active')
        read_only_fields = ('id', 'username', 'date_joined')

```

Fichier : `apps/settings/views.py`

Modifier UserSettingsDetailView (ligne 118-141) :

```

# AVANT (imports en haut du fichier)
from apps.authentication.serializers import UserProfileSerializer, UserRegistrationSerializer

# APRES
from apps.authentication.serializers import (
    UserProfileSerializer,
    UserRegistrationSerializer,

```

```

        AdminUserUpdateSerializer,
    )

    # AVANT (dans UserSettingsDetailView.patch)
    def patch(self, request, user_id):
        try:
            user = HollyUser.objects.get(id=user_id)
            serializer = UserProfileSerializer(user, data=request.data, partial=True)
            ...

    # APRES
    def patch(self, request, user_id):
        try:
            user = HollyUser.objects.get(id=user_id)
            serializer = AdminUserUpdateSerializer(user, data=request.data, partial=True)
            ...

```

4. Ajouter l'email dans la reponse employes (P1)

Fichier : `apps/staff/serializers.py` (ou le fichier contenant EmployeeSerializer)

Ajouter un champ email read-only :

```

class EmployeeSerializer(serializers.ModelSerializer):
    type_employe_name = serializers.CharField(
        source='type_employe.nom_type', read_only=True
    )
    # AJOUTER :
    email = serializers.SerializerMethodField()

    def get_email(self, obj):
        """Retourne l'email du user lie, ou None si pas de compte."""
        return obj.user.email if obj.user else None

    class Meta:
        model = Employe
        fields = (
            'id', 'user_id', 'last_name', 'first_name',
            'type_employe_id', 'type_employe_name',
            'salary', 'hire_date', 'phone_number',
            'email', # AJOUTER
        )

```

5. Endpoints reset/change password (P2)

Fichier : `apps/authentication/views.py`

Ajouter les views suivantes :

```

from django.contrib.auth.tokens import default_token_generator
from django.utils.http import urlsafe_base64_encode, urlsafe_base64_decode
from django.utils.encoding import force_bytes, force_str

class ChangePasswordView(APIView):

```

```
"""POST /api/auth/change-password/ -- User connecte change son password."""
permission_classes = [IsAuthenticated]
```

```
def post(self, request):
    old_password = request.data.get('old_password')
    new_password = request.data.get('new_password')
    new_password2 = request.data.get('new_password2')

    if not request.user.check_password(old_password):
        return Response(
            {'old_password': ['Mot de passe actuel incorrect.']},
            status=400
        )
    if new_password != new_password2:
        return Response(
            {'new_password': ['Les mots de passe ne correspondent pas.']},
            status=400
        )

    try:
        validate_password(new_password, request.user)
    except Exception as e:
        return Response({'new_password': list(e.messages)}, status=400)

    request.user.set_password(new_password)
    request.user.save()
    return Response({'message': 'Mot de passe modifie avec succes.'})
```

```
class ForgotPasswordView(APIView):
```

```
"""POST /api/auth/forgot-password/ -- Demande de reset (envoie un email)"""
permission_classes = [] # Public
```

```
def post(self, request):
    email = request.data.get('email')
    if not email:
        return Response({'email': ['Email requis.']}, status=400)

    # Toujours repondre 200 pour ne pas reveler si le compte existe
    try:
        user = User.objects.get(email=email)
        token = default_token_generator.make_token(user)
        uid = urlsafe_base64_encode(force_bytes(user.pk))
        # TODO: Envoyer l'email avec le lien de reset
        # reset_url = f"{settings.FRONTEND_URL}/reset-password?uid={uid}&token={token}"
        # send_mail(...)
    except User.DoesNotExist:
        pass

    return Response({
        'message': 'Si un compte existe avec cet email, un lien de reinitialisation a ete envoye.'
    })
```

```
class ResetPasswordView(APIView):
```

```
"""POST /api/auth/reset-password/ -- Confirmation du reset avec token."""
permission_classes = [] # Public
```

```
def post(self, request):
    uid = request.data.get('uid')
    token = request.data.get('token')
    new_password = request.data.get('password')
    new_password2 = request.data.get('password2')

    if new_password != new_password2:
        return Response(
            {'password': ['Les mots de passe ne correspondent pas.']},
            status=400
        )

    try:
```

```

        user_id = force_str(unsafe_base64_decode(uid))
        user = User.objects.get(pk=user_id)
    except (TypeError, ValueError, User.DoesNotExist):
        return Response({'error': 'Lien invalide.'}, status=400)

    if not default_token_generator.check_token(user, token):
        return Response({'error': 'Lien expire ou invalide.'}, status=400)

    try:
        validate_password(new_password, user)
    except Exception as e:
        return Response({'password': list(e.messages)}, status=400)

    user.set_password(new_password)
    user.save()
    return Response({'message': 'Mot de passe reinitialise avec succes.'})

```

Fichier : `apps/authentication/urls.py`

Ajouter les routes (dans urlpatterns) :

```

# Ajouter les imports
from .views import (
    ...
    ChangePasswordView,
    ForgotPasswordView,
    ResetPasswordView,
)

# Ajouter dans urlpatterns :
re_path(r'^change-password/?$', ChangePasswordView.as_view(), name='change_password'),
re_path(r'^forgot-password/?$', ForgotPasswordView.as_view(), name='forgot_password'),
re_path(r'^reset-password/?$', ResetPasswordView.as_view(), name='reset_password'),

```

6. Configuration email (P2)

Fichier : `holly_pi/settings.py`

Ajouter la configuration email :

```

# ---- Email Configuration ----
# Pour le developpement : affiche les emails dans la console
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'

# Pour la production : commenter et configurer
# EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
# EMAIL_HOST = os.environ.get('EMAIL_HOST', 'smtp.sendgrid.net')
# EMAIL_PORT = int(os.environ.get('EMAIL_PORT', 587))
# EMAIL_USE_TLS = True
# EMAIL_HOST_USER = os.environ.get('EMAIL_HOST_USER', '')
# EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_HOST_PASSWORD', '')

DEFAULT_FROM_EMAIL = os.environ.get(
    'DEFAULT_FROM_EMAIL', 'Holy Fork <noreply@hollyfork.fr>'
)
FRONTEND_URL = os.environ.get('FRONTEND_URL', 'http://localhost:5173')

```

Resume des fichiers a modifier

#	Fichier	Modification	Priorite
1	apps/settings/views.py	Permission MANAGE_STAFF sur POST, filtrage GET par restaurant	P0
2	apps/authentication/serializers.py	restaurant_id optionnel dans UserRegistrationSerializer	P0
3	apps/authentication/serializers.py	Ajouter AdminUserUpdateSerializer (is_active)	P1
4	apps/settings/views.py	Utiliser AdminUserUpdateSerializer dans PATCH	P1
5	apps/staff/serializers.py	Ajouter champ email dans EmployeeSerializer	P1
6	apps/authentication/views.py	Ajouter 3 views (change/forgot/reset password)	P2
7	apps/authentication/urls.py	Ajouter 3 routes password	P2
8	holly_pi/settings.py	Configuration EMAIL_BACKEND	P2

Aucune migration Django n'est necessaire -- toutes les modifications concernent des serializers, views, et configuration. Les modeles ne changent pas.

Ordre d'implementation recommande

1. Modifier serializers.py (restaurant_id optionnel) -- 5 min
2. Modifier settings/views.py (permissions + filtrage) -- 10 min
3. Tester POST /api/settings/users/ depuis le frontend -- verifier que l'invitation fonctionne
4. Ajouter AdminUserUpdateSerializer + email dans employees -- 10 min
5. Ajouter les endpoints password -- 20 min
6. Configurer email backend -- 5 min